GUIDE 06

GitHub Repositories, Version Control and Team Practices

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Create separate frontend and backend repositories.
- Use a safe feature-branch and pull-request workflow.
- Apply meaningful commits, reviews, tags and ignore rules.
- Prevent credentials, generated files and local configuration from entering Git history.

Prerequisites

- Git installed and configured.
- Frontend and backend projects exist locally.
- GitHub account with two-factor authentication recommended.

Suggested student time: 1.5-2 hours

1. Repository plan

Repository	Contains	Does not contain
unimart-frontend	React source, package-lock.json, tests, Dockerfile, frontend workflow, .env.example	node_modules, .env.local, built dist, secrets.
unimart-backend	Spring source, Maven Wrapper, migrations, tests, Dockerfile, backend workflow, .env.example	target, IDE files, real DB/JWT secrets, private keys.
Optional unimart-docs	Architecture diagrams, API contract, lab evidence	Passwords, personal student data, production exports.

2. Configure Git identity

```
[Terminal]
git config --global user.name "Your Name"
git config --global user.email "your-verified-email@example.com"
git config --global init.defaultBranch main
git config --list
```

3. Initialize and push each local project

```
[Terminal]
cd unimart-frontend
git init -b main
git add .
git commit -m "chore: initialize React TypeScript frontend"
git remote add origin <FRONTEND-REMOTE-URL>
git push -u origin main

cd ../unimart-backend
git init -b main
git add .
git commit -m "chore: initialize Spring Boot backend"
git remote add origin <BACKEND-REMOTE-URL>
git push -u origin main
```

Secret safety: GitHub explicitly warns against adding passwords, API keys or other sensitive information. Review git status and git diff --staged before every commit.

4. Required ignore files

```
[Frontend .gitignore]
# Frontend .gitignore
node_modules/
dist/
.env
.env.*
!.env.example
*.local
.vscode/
.DS_Store
npm-debug.log*
```

```
[Backend .gitignore]
# Backend .gitignore
target/
.idea/
*.iml
.env
.env.*
!.env.example
application-secrets.yml
*.pem
*.key
logs/
.DS_Store
```

5. Feature-branch workflow

```
[Terminal]
git switch main
git pull --ff-only
git switch -c feature/listing-create-api

# Work in small increments
git status
git diff
git add src/main/java/... src/test/java/...
git commit -m "feat(listings): add create listing endpoint"

git push -u origin feature/listing-create-api
```

1. Open a pull request into main.
2. Describe the requirement, implementation, screenshots/API evidence, tests and any migration.
3. Assign at least one reviewer. The author does not approve their own work.
4. Resolve review comments with new commits; avoid hiding significant changes in force-pushed history.
5. Merge only when CI passes. Delete the feature branch after merge.

6. Commit message convention

Prefix	Use	Example
feat	New user-visible behavior	feat(reviews): add review update form
fix	Bug correction	fix(auth): reject expired access token
test	Tests only	test(listings): cover unauthorized update
docs	Documentation	docs(api): document review endpoints
refactor	Behavior-preserving restructuring	refactor(listings): extract mapper
chore	Tooling/configuration	chore: add backend CI workflow
ci	Pipeline change	ci: publish backend image to GHCR

7. Sync and conflict resolution

```
[Terminal]
git fetch origin
git rebase origin/main
# Resolve files, then:
git add <resolved-file>
git rebase --continue
# Abort safely if necessary:
git rebase --abort
```

- Read both versions and the business intent; do not blindly choose “ours” or “theirs”.
- Run tests after resolution.
- For student teams, rebase local feature branches; avoid rewriting shared main history.
- Use pull --ff-only on main to prevent accidental merge commits.

8. Protect main and configure collaboration

- Require a pull request before merging.
- Require status checks for frontend build/lint and backend tests.
- Require conversation resolution and at least one approval where plan support permits.
- Restrict force pushes and deletion of main.
- Enable Dependabot/security alerts if available and review updates rather than merging automatically.
- Use Issues or a small project board to assign Listing, Review, Auth and Deployment tasks.

9. Releases and rollback points

```
[Terminal]
git switch main
git pull --ff-only
git tag -a v0.1.0 -m "Local CRUD milestone"
git push origin v0.1.0

git tag -a v1.0.0 -m "Cloud deployment milestone"
git push origin v1.0.0
```

10. Best-practice review checklist

- One pull request addresses one coherent change.
- Database migration and entity mapping are reviewed together.
- API contract changes are reflected in frontend types and Postman collection.
- Tests cover happy path, validation, unauthorized, forbidden and not-found cases.
- No generated directories, secrets or binary database dumps are included.
- README setup instructions work on a clean clone.

Completion checklist

☐ Frontend and backend are in separate GitHub repositories.

☐ Both repositories have correct .gitignore and .env.example files.

☐ main is protected from direct unreviewed changes.

☐ A feature branch and pull request have been completed.

☐ CI checks are planned/required before deployment.

☐ A milestone tag has been created.